

bundle - minify css,jss auto

cara biar otomatis ngebundle app, config, mic, tts, ui js gua jadi 1 file dan minify ketika gua save atau ada perubahan gimana ya biar ngg report manual

Biar lu nggak capek *save*, *copy-paste*, dan nge-minify manual terus-terusan, kita bisa bikin sistem **Auto-Bundle & Minify** yang jalan otomatis di belakang layar.

Karena kodingan lu sekarang pakai Vanilla JS (nggak pakai `import/export` ala React), kita **nggak perlu** pakai alat yang ribet kayak Webpack atau Vite yang malah berisiko ngerusak variabel global lu.

Kita bakal pakai kombinasi 2 alat murni dari Node.js yang sangat ringan:

1. **Terser**: Buat nggabungin (bundle) dan nge-minify file lu jadi satu.
2. **Nodemon**: Buat mantau (watch) kalau lu nge-save file, dia bakal otomatis nyuruh Terser kerja.

Berikut cara *setup*-nya yang cuma butuh waktu 2 menit:

Langkah 1: Install Alatnya

Buka Terminal / CMD di dalam folder *project* lu, lalu ketik ini:

Bash

```
npm install --save-dev terser nodemon
```

Langkah 2: Buat Script Otomatis di `package.json`

Buka file `package.json` lu, cari bagian `"scripts"`, lalu tambahkan dua baris ini:

JSON

```
"scripts": {  
  "start": "node server.js",  
  "build:js": "terser config.js ui.js mic.js tts.js app.js -o bundle.min.js -c -m",  
}
```

```
"watch:js": "nodemon --ext js --ignore bundle.min.js --exec \"npm run build:js\""
}
```

Penjelasan penting:

- Di bagian `"build:js"`, **urutan file sangat ngaruh!** Pastikan file yang jadi pondasi (seperti `config.js` atau `ui.js`) ditaruh duluan sebelum `app.js`. Sesuaikan nama file dan urutannya dengan kodingan lu.
- `o bundle.min.js` artinya hasil gabungannya akan disimpan ke file baru bernama `bundle.min.js`.
- `c -m` artinya *compress* dan *mangle* (nama variabel diubah jadi satu huruf biar sizenya super kecil).
- `-ignore bundle.min.js` ini wajib biar Nodemon nggak *looping* tanpa henti saat filenya selesai dibuat.

Langkah 3: Ubah HTML Lu

Sekarang buka file `index.html` lu. Hapus semua `<script>` lama yang berderet itu, dan ganti jadi satu file aja:

HTML

```
<script src="bundle.min.js"></script>
```

Langkah 4: Jalankan Mesin Otomatisnya!

Di terminal lu, jalankan perintah ini (biarkan terminalnya tetap terbuka saat lu ngoding):

Bash

```
npm run watch:js
```

Selesai! Sekarang cara kerjanya gini:

Setiap kali lu ngedit `app.js` atau `tts.js` dan nekan `Ctrl+S` (Save), Nodemon bakal langsung sadar dan nembak Terser. Dalam hitungan milidetik, file `bundle.min.js` lu bakal ter-update dengan kode yang udah digabung dan di-minify.

Lu tinggal *refresh* browser, dan aplikasinya udah pakai kode terbaru dengan ukuran file yang sekecil mungkin! Nggak ada lagi kerja manual, Ky.

Buat Script Otomatis di package.json

ini ditaro dimana package.json ?

File `package.json` itu harusnya berada tepat di **folder utama (root directory)** dari *project* lu.

Biar gampang bayanginnya, file ini harus **sejajar** (satu lokasi) dengan file `server.js` (atau file backend lu), `index.html`, dan folder tempat lu nyimpen kodingan JS lu.

Skenario 1: Kalau filenya BELUM ADA

Kalau pas lu cek di folder project ternyata lu nggak nemu file `package.json`, berarti project lu belum di-inisialisasi sebagai project Node.js. Bikinnya gampang banget:

1. Buka Terminal atau CMD.
2. Arahkan ke folder project lu (contoh: `cd C:\project-antrean`).
3. Ketik perintah ini lalu tekan Enter:

Bash

```
npm init -y
```

4. *Bim salabim!* File `package.json` bakal otomatis tercipta di folder itu.

Skenario 2: Cara masukin script-nya

Buka file `package.json` tersebut pakai *code editor* lu (misalnya VS Code). Isinya itu format JSON.

Cari bagian yang tulisannya `"scripts": { ... }`, lalu **timpa bagian itu** dengan script yang udah gua kasih sebelumnya.

Bentuk *full* dari file `package.json` lu nanti kira-kira bakal jadi seperti ini:

JSON

```
{
  "name": "aplikasi-antrean",
  "version": "1.0.0",
  "description": "Sistem antrean",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "build:js": "terser config.js ui.js mic.js tts.js app.js -o bundle.min.js -c -m",
    "watch:js": "nodemon --ext js --ignore bundle.min.js --exec \"npm run build:js\""
  },
  "keywords": [],
  "author": "Rizky",
  "license": "ISC",
  "devDependencies": {
    "nodemon": "^3.0.0",
    "terser": "^5.0.0"
  }
}
```

(Catatan: Jangan lupa koma `,` di setiap akhir baris JSON biar nggak eror merah-merah).

Kalau file `package.json` -nya udah di-save dengan tambahan `"scripts"` tersebut, lu tinggal buka terminal di folder yang sama, lalu ketik `npm run watch:js` buat nyalain mesin auto-minify nya!

driver-call/

```
├── backend/
|   ├── main.py          # FastAPI server + Edge TTS
|   ├── requirements.txt  # Python dependencies
|   └── .env              # Konfigurasi port
├── frontend/
|   ├── index.html        # Entry point
|   └── assets/
|       ├── css/
|           ├── main.css    # Reset, variabel global
|           └── animations.css # Semua keyframes
```

```

|   |   | — header.css    # Breadcrumb, judul
|   |   | — cards.css     # Driver cards
|   |   | — controls.css  # Tombol, mic, settings
|   |   | — js/
|       | — ui.js         # DOM updates, wave, log
|       | — tts.js        # Edge TTS API calls
|       | — mic.js        # Speech recognition
|       | — app.js        # Logic utama, init
| — audio/                # Cache audio sementara
| — README.md

```

ini struktur folder gua gua taro package.json nya dimana

Melihat struktur folder lu yang udah sangat rapi dan terpisah antara backend (Python) dan frontend (HTML/JS), tempat paling ideal untuk menaruh `package.json` adalah di dalam folder `frontend/`.

Kenapa di situ? Biar urusan *tools* Node.js (untuk nge-bundle JS) nggak nyampur sama urusan backend Python lu.

Karena posisi file JS lu ada di dalam sub-folder `assets/js/`, script di dalam `package.json`-nya harus disesuaikan sedikit biar *Terser* tahu jalan menuju file-file tersebut.

Ini langkah tepatnya:

1. Masuk ke folder Frontend dan Inisialisasi

Buka Terminal/CMD lu, lalu arahkan masuk ke folder frontend:

Bash

```

cd frontend
npm init -y
npm install --save-dev terser nodemon

```

2. Isi `package.json` yang Baru

Buka file `package.json` yang baru muncul di dalam folder `frontend/` lu, lalu ubah bagian `"scripts"` -nya menjadi seperti ini (perhatikan penambahan path `assets/js/`):

JSON

```
{
  "name": "frontend-driver-call",
  "version": "1.0.0",
  "description": "Frontend bundling",
  "scripts": {
    "build:js": "terser assets/js/ui.js assets/js/tts.js assets/js/mic.js assets/js/app.js -o assets/js/bundle.min.js -c -m",
    "watch:js": "nodemon --watch assets/js --ext js --ignore assets/js/bundle.min.js --exec \"npm run build:js\""
  },
  "devDependencies": {
    "nodemon": "^3.0.0",
    "terser": "^5.0.0"
  }
}
```

(Urutan file JS di atas penting, sesuaikan kalau misalnya `app.js` butuh variabel dari `ui.js`, berarti `ui.js` harus ditulis duluan).

3. Sesuaikan `index.html` Lu

Buka file `frontend/index.html`, lalu ganti deretan pemanggilan script JS lama lu yang banyak itu menjadi cuma satu baris ini:

HTML

```
<script src="assets/js/bundle.min.js"></script>
```

Cara Kerjanya Sekarang:

Setiap kali lu mau ngoding dan nyalain mode otomatisnya, lu tinggal buka terminal di dalam folder `frontend/`, lalu jalankan:

Bash

```
npm run watch:js
```

Nodemon sekarang akan khusus memantau folder `assets/js/`. Tiap lu nge-save perubahan di `app.js` atau file JS lainnya, dia bakal langsung nggabungin semuanya jadi satu file ringkas `bundle.min.js` di dalam folder yang sama. Tampilan lu makin ngebut, dan foldernya tetep super rapi!

```
"main": "server.js"
```

ini server.js diset sesuai lokasinya?

Ah, lu teliti banget, Ky!

Jawabannya: **Hapus aja baris** `"main": "server.js"` **itu.**

Kenapa? Karena baris `"main"` itu sebenarnya cuma kepakai kalau lu bikin *library* (modul) Node.js untuk di-*publish* ke internet, atau kalau backend project tersebut murni pakai Node.js (misal pakai Express.js).

Berhubung struktur folder lu backend-nya pakai **Python** (`main.py`), dan `package.json` ini posisinya ada di folder `frontend/` cuma sebagai "alat bantu" buat nge-bundle JS, parameter `"main"` itu udah nggak ada fungsinya sama sekali. Nggak bakal ngaruh ke script Terser atau Nodemon yang lagi kita jalanin.

Jadi, bentuk file `package.json` lu di folder `frontend/` cukup dibikin bersih kayak gini aja:

JSON

```
{
  "name": "frontend-driver-call",
  "version": "1.0.0",
  "description": "Frontend bundling",
  "scripts": {
    "build:js": "terser assets/js/ui.js assets/js/tts.js assets/js/mic.js assets/js/app.js -o assets/js/bundle.min.js -c -m",
    "watch:js": "nodemon --watch assets/js --ext js --ignore assets/js/bundle.min.js --exec \"npm run build:js\""
  },
  "devDependencies": {
    "nodemon": "^3.0.0",
    "terser": "^5.0.0"
  }
}
```

Tinggal *save* JSON di atas, lalu jalanin `npm run watch:js`. Sistemnya bakal fokus mantau JS lu tanpa sibuk nyari file `server.js` yang emang nggak ada di folder *frontend* lu!

lokasi file js gua ada di `frontend/assets/js/`

Nah, mantap! Karena lu naruh file `package.json` -nya di dalam folder `frontend/`, berarti kodingan *path* yang gua kasih sebelumnya udah **100% pas dan langsung jalan**.

Karena posisi `package.json` -nya ada di `frontend/`, sistem otomatis tahu kalau tulisan `assets/js/` itu maksudnya adalah `frontend/assets/js/`.

Biar lu tinggal *copy-paste* dan nggak pusing, ini bentuk final `package.json` lu yang paling bersih:

JSON

```
{
  "name": "frontend-driver-call",
  "version": "1.0.0",
  "description": "Frontend JS Bundler",
  "scripts": {
    "build:js": "terser assets/js/ui.js assets/js/tts.js assets/js/mic.js assets/js/app.js -o assets/js/bundle.min.js -c -m",
    "watch:js": "nodemon --watch assets/js --ext js --ignore assets/js/bundle.min.js --exec \"npm run build:js\""
  },
  "devDependencies": {
    "nodemon": "^3.0.0",
    "terser": "^5.0.0"
  }
}
```

(Catatan: Kalau misal lu punya file JS tambahan kayak `config.js` atau `api.js`, tinggal selipin aja di bagian `build:js` sebelum `app.js` ya).

Cara Ngejalaninnya Biar Nggak Error:

Penting nih, pas lu mau nyalain mesin otomatisnya di Terminal/CMD, pastikan posisi *path* terminal lu udah **masuk ke dalam folder frontend**.

1. Buka Terminal.
2. Masuk ke folder frontend: `cd frontend` (atau arahkan sesuai lokasi lu).
3. Baru jalankan: `npm run watch:js`

Udah deh! Tiap kali lu nge-*save* file apapun di dalam folder `frontend/assets/js/`, otomatis bakal ke-bundle jadi `bundle.min.js` di folder yang sama.

Di `index.html` lu tinggal panggil `<script src="assets/js/bundle.min.js"></script>`, UI lu bakal tetep ngebut dan *source code* aslinya tetep rapi per-file! Lanjut ngodingnya, Ky!

ini jadi setiap ada perubahan file otomatis terupdate kan yan bundle nya?

Betul banget, Ky! 100% otomatis.

Selama terminal tempat lu ngetik `npm run watch:js` itu **tetap terbuka dan menyala di background**, sistemnya bakal terus berjaga-jaga seperti CCTV.

Jadi alur kerja lu sekarang bakal se-simpel ini:

1. Lu ngedit kodingan di `app.js` atau file JS lainnya.
2. Lu tekan **Ctrl + S** (Save).
3. Dalam hitungan milidetik, **Nodemon** langsung sadar ada file yang berubah.
4. Nodemon otomatis nyuruh **Terser** buat ngegabungin dan nge-minify semuanya ke `bundle.min.js` saat itu juga.
5. Lu tinggal *refresh* browser, dan perubahan terbarunya udah langsung jalan.

Nggak ada lagi capek-capek ketik *command* atau *copy-paste* manual. Kalau mau berhenti atau udahan ngodingnya, lu tinggal ke terminal itu terus tekan **Ctrl + C** buat matiin mesin *watch*-nya.

Cobain deh tes ubah satu baris `console.log` di `app.js`, terus *save*, ntar di terminal bakal kelihatan Nodemon me- *restart* dan nge- *build* ulang secara otomatis!

buatin yang buat css nya juga

Bisa banget, Ky! Biar sekalian rapi, kita jadiin satu pintu aja mesin otomatisnya. Jadi pas lu nge-*save* file JS **ataupun** file CSS, sistem bakal otomatis nge-bundle dan nge-minify dua-duanya secara bersamaan.

Biar CSS lu bisa di-minify, kita butuh satu alat tambahan yang super ringan dari Node.js namanya `clean-css-cli`.

Ikuti 3 langkah gampang ini:

Langkah 1: Install Alat Tambahan untuk CSS

Pastikan lu masih di dalam folder `frontend/` di terminal lu, lalu ketik ini buat nginstall `clean-css` :

Bash

```
npm install --save-dev clean-css-cli
```

Langkah 2: Update `package.json` Lu

Sekarang kita update script-nya biar dia bisa nangani JS dan CSS sekaligus. Buka `package.json` lu, dan ganti kodenya jadi seperti ini:

JSON

```
{
  "name": "frontend-driver-call",
  "version": "1.0.0",
  "description": "Frontend JS & CSS Bundler",
  "scripts": {
    "build:js": "terser assets/js/ui.js assets/js/tts.js assets/js/mic.js assets/js/app.js -o assets/js/bundle.min.js -c -m",
    "build:css": "cleancss -o assets/css/bundle.min.css assets/css/main.css assets/css/animations.css assets/css/header.css assets/css/cards.css assets/css/controls.css",
    "build": "npm run build:js && npm run build:css",
    "watch": "nodemon --watch assets --ext js,css --ignore assets/js/bundle.min.js --ignore assets/css/bundle.min.css --exec \"npm run build\""
  },
  "devDependencies": {
    "clean-css-cli": "^5.6.3",
    "nodemon": "^3.0.0",
    "terser": "^5.0.0"
  }
}
```

Apa yang berubah di sini?

1. Ada `"build:css"` : Ini tugasnya ngambil semua file CSS lu (urutannya dari `main.css` yang isinya variabel global, sampai ke komponen-komponennya), lalu digabung jadi satu ke `assets/css/bundle.min.css` .
2. Ada `"build"` : Ini adalah komandan utamanya. Dia nyuruh mesin nge-build JS, terus dilanjut nge-build CSS.
3. Di `"watch"` : Sekarang Nodemon mantau folder `assets` secara keseluruhan, dan ngecek ekstensi file `-ext js,css` . Jadi kalau ada perubahan di JS atau CSS, dia langsung lari ngejalanin perintah `"build"` .

Langkah 3: Ubah `<link>` CSS di HTML Lu

Buka `index.html` , hapus semua deretan `<link rel="stylesheet">` lu yang panjang itu, dan ganti jadi satu baris aja di bagian `<head>` :

HTML

```
<link rel="stylesheet" href="assets/css/bundle.min.css">
```

Cara Pakenya:

Sama persis kayak sebelumnya! Di terminal (di dalam folder `frontend`), lu tinggal jalanin:

Bash

```
npm run watch
```

Sekarang, sistem lu udah level dewa, Ky! Lu mau ngedit desain di `cards.css` kek, atau ngedit logika antrean di `app.js` kek, tinggal **Ctrl + S**. Di balik layar, file-file lu bakal otomatis dijahit dan di-press jadi sekecil mungkin. Web antrean lu makin rapi dan ngebut banget *loading*-nya!